

RUES ANÁLISIS

Pablo Ibarlucea González y Oscar Javier Bachiller Sandoval

<https://github.com/pabloibargon/rues-analysis>

- **Integración:** Enriquecer los datos registrales con contexto macroeconómico (Banco Mundial).
- **Limpieza:** Selección de variables y transformación a tipos adecuados.
- **Análisis No Supervisado:** Segmentación de empresas (Clustering).
- **Análisis Supervisado:** Predicción del tipo de sociedad (Clasificación).
- **Validación:** Contraste de hipótesis geográfico.

python

```
1 # Definimos columnas con baja cardinalidad para optimizar memoria
2 category_cols = ["Estado de la matrícula", "Tipo de Sociedad", ...]
3 df[category_cols] = df[category_cols].astype("category")
4 # ...
5 # Conversión de fechas
6 date_cols = ["Fecha de Matrícula", "Fecha de Vigencia", ...]
7 df[date_cols] = df[date_cols].apply(pd.to_datetime, errors="coerce")
8 # ...
9 # Mapeo manual de booleanos y eliminación de ruido
10 bool_cols = ["Emprendimiento Social", "Extinción de Dominio"]
11 df[bool_cols] = df[bool_cols].apply(lambda s: s.map({"S": True, "N": False}))
12 # Elimina fecha de cancelacion por alto numero de nulos e identificadores
13 df = df.drop(columns=["Fecha de Cancelación", "Número de Matrícula", ...])
```

python

```
1 # Descarga y lectura directa del ZIP desde la API
2 url = "https://api.worldbank.org/v2/en/country/COL?downloadformat=csv"
3 response = requests.get(url)
4 # ... (Extracción del ZIP en memoria) ...
5 wb_source = pd.read_csv(f, skiprows=4)
6
7 target_indicators = {
8     'NY.GDP.MKTP.KD.ZG': 'PIB_Crecimiento',
9     'FP.CPI.TOTL.ZG': 'Inflacion', ...
10 }
11 # ... (Filtrado y limpieza de columnas del BM) ...
12 # Transformación: De formato 'Ancho' a 'Largo' y luego Pivot
13 wb_melted = wb_clean.melt(id_vars=['Indicador'], var_name='Year', ...)
14 wb_pivot = wb_melted.pivot(index='Year', columns='Indicador', values='Valor')
15 # Integración: Contexto de nacimiento (Left Join por Año Matrícula)
16 df = df.merge(wb_pivot.add_suffix('_Matricula'), on='Year_Matricula', how='left')
```

Fuente Externa: API del Banco Mundial.

Variables Clave Agregadas:

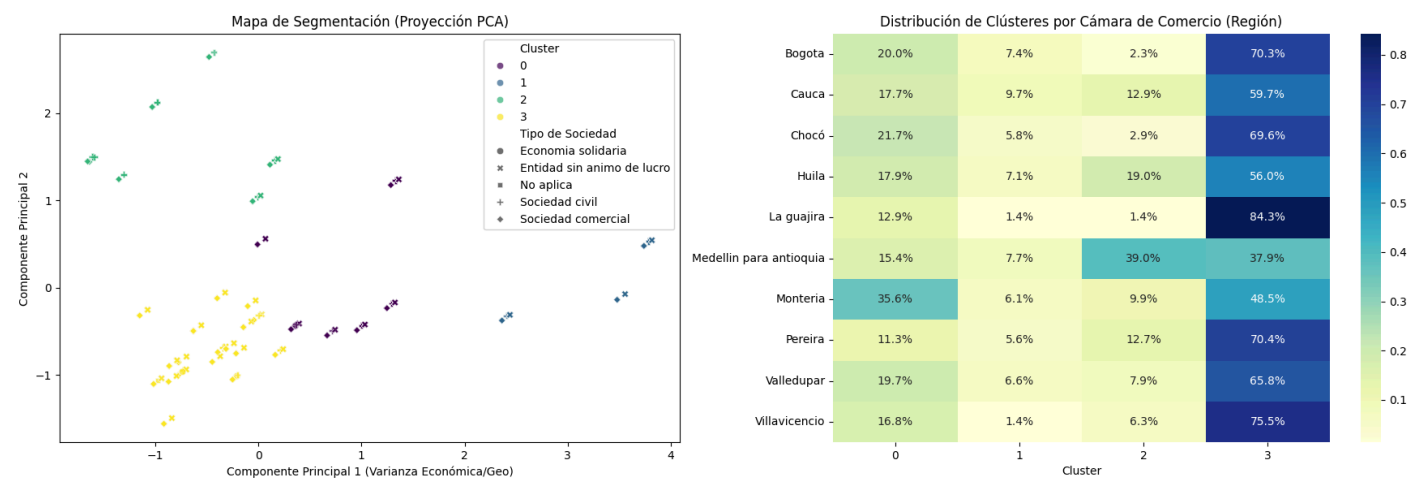
- Crecimiento del PIB anual.
- Inflación (IPC).
- Tasa de desempleo.

Estrategia: Left Join utilizando el **Año de Matrícula**.

python

```
1 # ... (Definición de num_features y cat_features) ...
2 # Preprocesamiento diferenciado numérico/categorico
3 preprocessor = ColumnTransformer(transformers=[
4     ('num', StandardScaler(), num_features),
5     ('cat', OneHotEncoder(), cat_features)
6 ])
7 # Pipeline: Limpieza -> PCA (Reducción) -> Modelo
8 pipeline = Pipeline([
9     ('preprocessor', preprocessor),
10    ('pca', PCA(n_components=2)),
11    ('kmeans', KMeans(n_clusters=4, random_state=44))
12 ])
13 pipeline.fit(df_model)
14 # Guardamos coordenadas PCA para visualización
15 df_model['Cluster'] = pipeline.named_steps['kmeans'].labels_
```

Modelo: K-Means (k=4) + PCA.

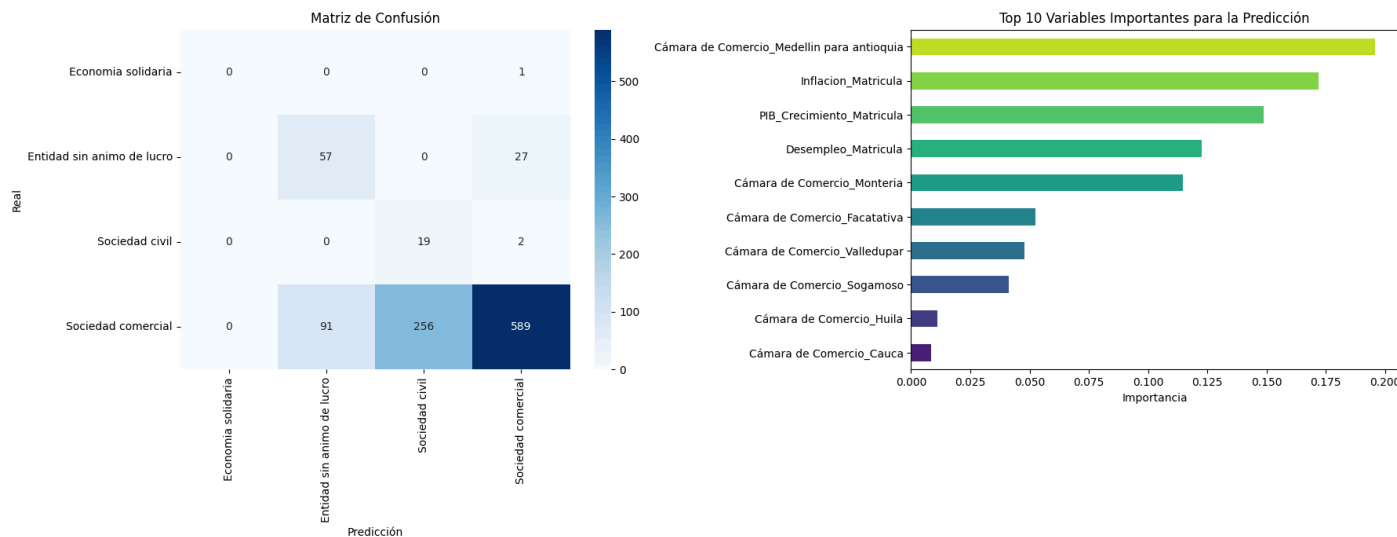


Conclusiones: Existencia de un “Cluster Base” (70% de datos) y una anomalía estructural en **Antioquia** (Cluster 2).

python

```
1 # Eliminar clases con < 2 registros (rompen el stratify)
2 v_counts = df_clf[target_col].value_counts()
3 df_clf = df_clf[df_clf[target_col].isin(v_counts[v_counts >= 2].index)]
4 # ... (Preprocesamiento con ColumnTransformer) ...
5 # Split estratificado
6 X_train, X_test, y_train, y_test = train_test_split(..., stratify=y)
7 # Random Forest con pesos balanceados para mitigar desbalanceo
8 model_rf = RandomForestClassifier(
9     n_estimators=200,
10     class_weight="balanced", # CRÍTICO: Penaliza error en clases minoritarias
11     n_jobs=-1
12 )
13 model_rf.fit(X_train, y_train)
```


Modelo: Random Forest Classifier.

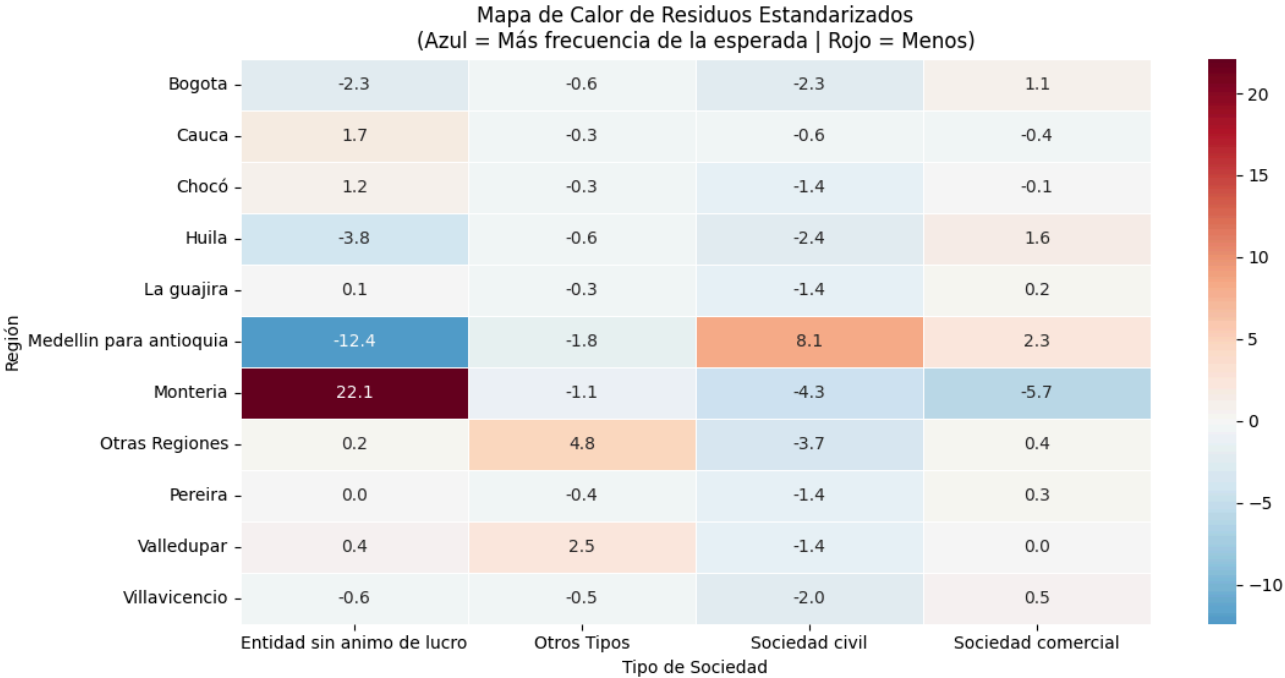


Conclusiones: La ubicación (Cámara de Comercio) y la **Inflación** al momento de registro son los predictores más potentes.

python

```
1 # Agrupación de colas largas para validez estadística
2 top_camaras = df['Cámara'].value_counts().nlargest(10).index
3 df['Region_Agrupada'] = df['Cámara'].apply(
4     lambda x: x if x in top_camaras else 'Otras Regiones'
5 )
6 # Agrupar Tipos de Sociedad: Mantener los que tengan > 1% de datos, resto a 'Otros'
7 threshold = len(df_chi) * 0.01
8 soc_counts = df_chi['Tipo de Sociedad'].value_counts()
9 valid_socs = soc_counts[soc_counts > threshold].index
10 df_chi['Sociedad_Agrupada'] = df_chi['Tipo de Sociedad'].apply(lambda x: x if x in
    valid_socs else 'Otros Tipos')
11 # ... (Creación de tabla de contingencia) ...
12 # Test estadístico
13 chi2, p_val, dof, expected = stats.chi2_contingency(contingency_table)
14 # Cálculo de Residuos Estandarizados (Para el Heatmap)
15 residuals = (contingency_table - expected) / np.sqrt(expected)
```

Prueba: Chi-Cuadrado de Pearson. **Hipótesis:** Dependencia entre Región y Tipo de Sociedad.



Resultado: $p < 0.05$. Se rechaza la independencia. Montería y Medellín muestran desviaciones significativas del comportamiento nacional.

- **Éxito en la Integración:** Las variables macroeconómicas demostraron ser señales predictivas reales, no ruido.
- **Singularidad Regional:** El tejido empresarial no es homogéneo; Antioquia y Montería requieren análisis diferenciados.
- **Desafío de Datos:** El desbalanceo masivo hacia “Sociedad Comercial” (90%) es el principal reto técnico.

Gracias